

The 5-Layer System Design Framework

Use this order in every interview. Fill it in as you talk through the problem.

1

Requirements

What are you building, for how many users, and what matters most — reads, writes, or both?

- Functional requirements (what it must do)
- Non-functional requirements (scale, latency, availability)
- Clarifying questions to ask the interviewer

2

Data Model

What are you storing, and how is it structured?

- Core entities and relationships
- SQL vs NoSQL — and why
- Key fields / schema sketch

3

Scale & Traffic

Read-heavy or write-heavy? What's the peak load?

- Estimated QPS (queries per second)
- Storage growth over time
- Read:write ratio

4

Reliability & Caching

What happens when something fails? Where do you cache?

- Cache layer — what's cached, where, TTL
- Redundancy / failover plan
- Bottlenecks and single points of failure

5

Tradeoffs

Every choice costs something. Say it out loud.

- Consistency vs availability
- Cost vs performance
- What you'd change at 10x scale

Remember: interviewers aren't grading your answer. They're grading how you think.

[@techie_programmer](#)

Follow for more developer interview breakdowns